APM 2025-2026 连续考核 1：试题

整理日期：2026 年 10 月 18 日

目录

I 课程知识题 **2**
- Q.1 . . . 2
- Q.2 . . . 2
- Q.3 . . . 2
- Q.4 . . . 2
- Q.5 . . . 2

II 多个 kernel 的先后依赖 **2**
- Q.6 . . . 3

III Safe/Unsafe **3**
- Q.7 . . . 3
- Q.8 . . . 3
- Q.9 . . . 4
- Q.10 . . . 4

IV CUDA 求和归约 **5**
- Q.11 . . . 5

原卷注明时长：3 小时。教师：Julien Jaeger。所有答案必须解释理由，书面表达质量计入评价。四张照片中没有看到分值。

原题中的标识符、尺寸、指针偏移与 CUDA 调用照原样保留，即使其中有错。解析见参考解析。

I 课程知识题

Q.1

`cudaMallocHost` 做什么？它与 `cudaMalloc`、`malloc` 有什么区别？

Q.2

什么是合并访存（accès coalescé）？

Q.3

线程块为三维，大小分别为 `BLOCK_X`、`BLOCK_Y`、`BLOCK_Z`；网格为二维，大小分别为 `GRID_X`、`GRID_Y`。写出一个 CUDA 线程的全局索引计算式。

Q.4

下列哪个关键字不是 CUDA API 中的数据属性？

1. `__device__`
2. `__shared__`
3. `__local__`
4. `__constant__`

Q.5

OpenMP 指令 `#pragma omp target data map(to :a[0 :8])` 对应的 OpenACC 写法是什么？

II 多个 kernel 的先后依赖

同一个 GPU 上的 11 个 kernel 在 3 个不同 stream 中执行，依赖关系如图 1。箭头代表数据依赖，例如 A 产生的数据被 C 和 D 使用。

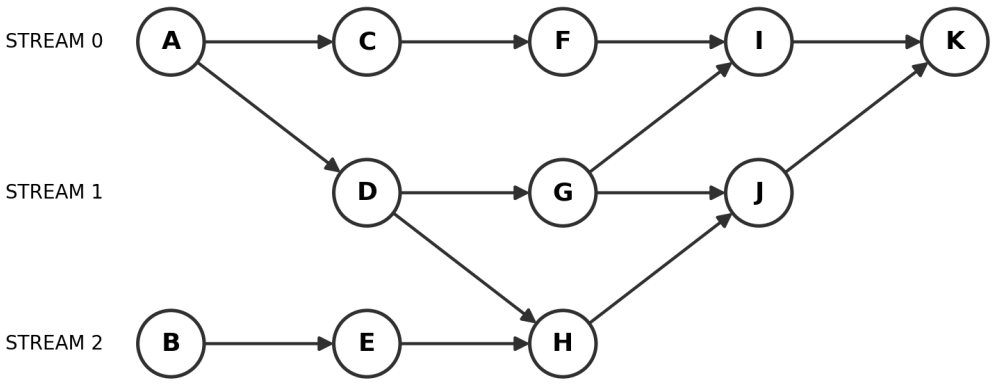


图 *1*：按原卷第 *2* 页重绘的依赖图。

Q.6

写出各 kernel 的调用代码，加入保证程序安全并得到正确结果所需的最少同步。

III Safe/Unsafe

判断以下 CUDA 代码片段是否可能导致错误或非确定性行为，并说明理由。

Q.7

代码如下：

```
cudaStream_t stream[1];
cudaStreamCreate(&stream[0]);
int *htab, *dtab, *tmp;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*size);
cudaMallocHost((void**)&tmp, size);
cudaMallocHost((void**)&htab, size);
memset(htab, 0, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice, stream[0]);
kernel<<<grid, block, 0, stream[0]>>>(dtab, tmp, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]); for(i=0; i<N; i++)
    printf("%d", htab[i]);
```

Q.8

代码如下：

```
cudaStream_t stream[3];
cudaStreamCreate(&stream[0]); cudaStreamCreate(&stream[1]);
cudaStreamCreate(&stream[2]);
int *htab, *dtab;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 6*size);
cudaMallocHost((void**)&htab, 7*size);
memset(htab, 0, 7*size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + size, htab + size, 2*size,
                cudaMemcpyHostToDevice, stream[1]);
cudaMemcpyAsync(dtab + 3*size, htab + 3*size, 3*size,
                cudaMemcpyHostToDevice, stream[2]);
kernel<<<grid0, block0, 0, stream[0]>>>(dtab, N);
kernel<<<grid1, block1, 0, stream[1]>>>(dtab + size, 2*N);
kernel<<<grid2, block2, 0, stream[2]>>>(dtab + 3*size, 3*N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]);
cudaMemcpyAsync(htab + size, dtab + size, 2*size,
                cudaMemcpyDeviceToHost, stream[1]);
cudaStreamSynchronize(stream[1]);
cudaMemcpyAsync(htab + 3*size, dtab + 3*size, 3*size,
                cudaMemcpyDeviceToHost, stream[2]);
cudaStreamSynchronize(stream[2]); for(i=0; i<7*N; i++)
    printf("%d", htab[i]);
```

Q.9

代码如下：

```
int *htab, *dtab;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*size);
cudaMallocHost((void**)&htab, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice);
kernel<<<grid, block>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost);
for(i=0; i<N-1; i++)
    htab[i] = htab[i+1];
htab[N-1] = 0;
```

Q.10

代码如下：

```
cudaStream_t stream[2];
cudaStreamCreate(&stream[0]); cudaStreamCreate(&stream[1]);
int *htab, *dtab;
int sz_in_bytes = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*sz_in_bytes);
cudaMallocHost((void**)&htab, 2*sz_in_bytes);
memset(htab, 0, 2*sz_in_bytes);
cudaMemcpyAsync(dtab, htab, sz_in_bytes,
                cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + sz_in_bytes, htab + sz_in_bytes, sz_in_bytes,
                cudaMemcpyHostToDevice, stream[1]);
kernel<<<grid, block, 0, stream[0]>>>(dtab + sz_in_bytes, N);
kernel<<<grid, block, 0, stream[1]>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, sz_in_bytes,
                cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]);
cudaMemcpyAsync(htab + sz_in_bytes, dtab + sz_in_bytes, sz_in_bytes,
                cudaMemcpyDeviceToHost, stream[1]);
cudaStreamSynchronize(stream[1]);
for(i=0; i<2*N; i++)
    printf("%d", htab[i]);
```

IV CUDA 求和归约

求和归约把数组所有元素相加，串行版本可以写成：

```
float sum = 0;
for (int i = 0; i < ntot; i++)
    sum += tab[i];
```

tab 已位于 GPU 上。分两阶段实现：先在每个块内归约，得到长度等于块数的部分和数组；再归约部分和数组，得到总和。

Q.11

实现 reduce_kernel(float *in, float *out)，计算每个线程块的部分和。in 的长度等于网格中启动的线程总数；out 存放每个线程块的部分和，长度等于线程块数。

给出该 kernel 的两次连续调用：第一次计算块内部分和，第二次对部分和归约，得到总和。

为简化问题，假设线程块数和每块线程数均为 2 的幂，原数组长度等于总线程数。